Meilenstein 04 – AuthKit: Integrationstest, Key-Handling und 401 bei ungültigem Refresh-Cookie

Feld	Wert
Projekt	Overex
Zeitraum	2026-10-11 – 2026-10-11 (laut Commits; begonnen wurde der Integrationstest laut Verlauf bereits am 2026-10-10)
Dokumentiert am	2026-10-12 05:56 CEST
Repository / Service	`authkit` (github.com/Umex10/authkit), Backend `apps/backend` (künftiger `auth-service` von Overex)
Thema	Vollständiger `@SpringBootTest` ohne lokale Key-Dateien, zentrales Lesen/Schreiben der RSA-Keys, automatische Dev-Keys beim ersten Start, Fix 500 → 401 beim Refresh, Doku auf RS256 umgestellt
Commits	8 (`045b703` .. `c54c3f6`), Branch `feature/integration-test` , lokal per `--no-ff` nach `main` gemergt (Merge-Commit `c54c3f6`)
Zeitaufwand	nicht erfasst
Bezug	Projektarbeit (5. Semester) und Grundlage für die Bachelorarbeit (6. Semester)

Ziel

Nach M02 gab es Unit- und Slice-Tests, aber keinen Test, der die ganze Anwendung zusammen startet. Ziel war ein Integrationstest, der den echten Ablauf eines Clients durchspielt (Sign-up → Sign-in → `/me` → Refresh) und prüft, ob ein anderer Microservice die Tokens mit dem veröffentlichten Public Key (JWKS) verifizieren kann. Das ist genau der Weg, den die übrigen Overex-Services später gehen.

Dabei kam eine zweite Frage auf: Ein Entwickler, der das Repo klont, hat keine Keys. Ziel war deshalb auch, dass AuthKit nach einem Clone ohne Handarbeit startet, in Produktion aber nie selbst Keys erzeugt.

Ausgangslage

- Der Integrationstest war als WIP vorhanden (Stash vom 2026-10-09, auf `feature/integration-test` wieder angewendet). Er scheiterte, weil der echte `KeyConfig` Key-Dateien aus `./keys/` lesen will, die es auf dem CI-Runner nicht gibt.
- `KeyConfig` las die PEM-Dateien selbst; die Test-Hilfsklasse `TestKeys` hatte eine eigene Kopie der PEM-Logik.
- Ohne `apps/backend/keys/` startete das Backend nicht. In `docs/RUNNING.md` standen nur drei `openssl` -Befehle.
- `backend-verify-ci` schloss die Integrationstests als „TEMPORARY“ aus.
- `SECURITY.md` , `ARCHITECTURE.md` und `BACKEND_FLOW.md` beschrieben noch den alten HS256-Ablauf mit eigenem `JwtAuthFilter` .

Umsetzung (chronologisch)

1. **2026-10-10** – Integrationstest gemeinsam mit Claude aufgebaut: `@TempDir + @DynamicPropertySource` schreiben ein Test-Key-Paar in einen Temp-Ordner und zeigen Spring diese Pfade (laut Verlauf, noch nicht committet).
2. **2026-10-11** – `RsaKeyFiles` als einzige Stelle für das PEM-Format eingeführt (lesen, schreiben, erzeugen); `KeyConfig` delegiert nur noch, `KeyConfigTest` wurde zu `RsaKeyFilesTest` (`045b703`).
3. **2026-10-11** – `DevKeyGenerator` hängt sich an `ApplicationEnvironmentPreparedEvent` und erzeugt beim Start ein Key-Paar, wenn beide Dateien fehlen. Schalter `jwt.generate-missing-keys` (`JWT_GENERATE_MISSING_KEYS`), im Dockerfile auf `false` (`e827d77`).
4. **2026-10-11** – `scripts/generate-jwt-keys.sh` für den Docker-Weg und Produktion, README-Schritt 0 und Abschnitt „JWT keys“ in `RUNNING.md` (`620e198`).
5. **2026-10-11** – Integrationstest vervollständigt: 7 Abläufe inkl. JWKS-Struktur, Verifikation eines Access-Tokens mit einem Decoder, der nur aus `jwks.json` gebaut wird, und Ablehnung eines Refresh-Tokens über den `type_tk` -Check; Admin-Testdaten in `TestDataFactory` (`fbe4fe7`).
6. **2026-10-11** – Nicht offensichtliche Stellen aller Tests kommentiert (`3aa8cb2`).
7. **2026-10-11** – Fix: Ein ungültiges Refresh-Cookie lieferte 500; `JwtService` übersetzt JJWT-Fehler jetzt in `BadCredentialsException` → 401 (`b80a366`).
8. **2026-10-11** – Doku auf den RS256-Resource-Server umgeschrieben, `CUSTOMIZING.md` und `TESTING.md` aktualisiert (`122d289`).
9. **2026-10-11** – Merge nach `main` (`c54c3f6`).

Geänderte Dateien

Datei	Art	Zweck
`apps/backend/src/main/java/.../auth/config/RsaKeyFiles.java`	neu	PEM lesen/schreiben, Key-Paar erzeugen; statische Hilfsklasse, auch ohne Spring nutzbar
`apps/backend/src/main/java/.../auth/config/DevKeyGenerator.java`	neu	Dev-Keys beim ersten Start (nur wenn beide Dateien fehlen und der Schalter an ist)

`apps/backend/src/main/java/.../auth/config/KeyConfig.java`	geändert	delegiert an `RsaKeyFiles`
`apps/backend/src/main/java/.../BackendApplication.java`	geändert	registriert `DevKeyGenerator` in `main`
`apps/backend/src/main/java/.../auth/JwtService.java`	geändert	JJWT-Fehler beim Refresh → `BadCredentialsException` (401)
`apps/backend/src/main/resources/application.properties`	geändert	`jwt.generate-missing-keys=${JWT_GENERATE_MISSING_KEYS:true}`
`apps/backend/Dockerfile`	geändert	`ENV JWT_GENERATE_MISSING_KEYS=false`
`scripts/generate-jwt-keys.sh`	neu	Key-Paar mit OpenSSL, überschreibt nie
`apps/backend/src/test/.../integration/BackendApplicationTests.java`	geändert	vollständiger Integrationstest inkl. JWKS-Verifikation und 401-Fall
`apps/backend/src/test/.../unit/auth/config/DevKeyGeneratorTest.java`	neu	8 Unit-Tests für den Generator (`MockEnvironment`, Event von Hand gebaut)
`apps/backend/src/test/.../unit/auth/config/RsaKeyFilesTest.java`	umbenannt (aus `KeyConfigTest`)	Lese-Tests plus Schreib-/Lese-Rundlauf
`apps/backend/src/test/.../utils/TestKeys.java`, `TestDataFactory.java`	geändert	nutzen `RsaKeyFiles`; Admin-Sign-in-Daten
`apps/backend/src/test/...` (AuthControllerTest, JwksControllerTest, MeControllerTest, AuthServiceTest, JwtServiceTest)	geändert	Kommentare; JwtServiceTest prüft die neue Exception samt Ursache
`README.md`, `docs/RUNNING.md`	geändert	Schritt 0 mit Skript, Dev- vs. Produktions-Keys
`docs/SECURITY.md`, `ARCHITECTURE.md`, `BACKEND_FLOW.md`, `CUSTOMIZING.md`, `TESTING.md`	geändert	RS256-Resource-Server, JWKS-Ablauf für andere Services, aktuelle Tests

Entscheidungen

Entscheidung	Verworfene Alternativen	Begründung
Keys im Test über `@TempDir` + `@DynamicPropertySource`	lokalen `keys/`-Ordner voraussetzen; `DevKeyGenerator` im Test nutzen	Test läuft auf jedem Rechner und in der CI gleich und verändert keine Dateien außerhalb des Tests
Dev-Keys per `ApplicationListener<ApplicationEnvironmentPreparedEvent>`	`CommandLineRunner`; `@Configuration`-Bean	Runner läuft erst nach dem Context-Start, `KeyConfig` braucht die Dateien aber schon beim Bauen der Beans; in Phase 2 existieren noch keine Beans
Listener in `main` registriert	`spring.factories`	`@SpringBootTest` ruft `main` nicht auf, dadurch erzeugen Tests nie Dev-Keys
Erzeugen nur, wenn **beide** Dateien fehlen	fehlende Datei ergänzen	ein neuer Key neben einem alten bildet kein Paar
Schalter: Default `true` in den Properties, `false` im Code, `false` im Dockerfile	immer erzeugen	lokal bequem, in Produktion fehlen Keys laut statt still mit Wegwerf-Key
`RsaKeyFiles` statisch	Bean / `@Configuration`	keine Felder, muss vor dem Context und in Unit-Tests ohne Spring nutzbar sein
Dev-Keys mit Standardrechten (`rw-r--r--`)	nur Besitzer (`rw-------`)	sonst kann der Container-User `authkit` (uid 1001) die gemounteten Keys nicht lesen
`TestKeys` und `PublicKeyConfig` behalten	`@Import(KeyConfig)` im Slice-Test	Slice-Tests sollen ohne Dateien laufen; Signieren und Prüfen nutzen garantiert dasselbe Paar
JWKS-Decoder im Test aus dem JSON gebaut	`JwtDecoder`-Bean injizieren	die Bean bekommt den Key direkt aus `KeyConfig` und würde einen kaputt veröffentlichten Key nicht bemerken
Fix im `JwtService` (Exception übersetzen)	eigener Handler für `JwtException`	gleicher Weg wie der bestehende `type_tk`-Check; der Service wirft nur Spring-`AuthenticationException`s
Produktions-Compose nicht im Repo	Compose mit `secrets:` auf Verdacht	hängt von der Plattform ab (Railway, Docker-Secrets, Kubernetes); Ablauf in `RUNNING.md` beschrieben

Neu gelernt

- **`@TempDir` / `@DynamicPropertySource`** : JUnit legt einen leeren Ordner an; Spring Test lässt mich Properties vor dem Context-Start setzen, und diese Werte gewinnen gegen die Properties-Dateien.
- **Profile ergänzen statt ersetzen**: `@ActiveProfiles("test")` liest zuerst `application.properties` und legt `application-test.properties` darüber.
- **Startphasen von Spring Boot**: Starting → EnvironmentPrepared (Properties gelesen, noch keine Beans) → Context → Refresh (Beans entstehen) → Started (`CommandLineRunner`) → Ready.
- **Statische Methoden vs. Objekt**: Statisch, wenn sich die Klasse nichts merkt; Objekt/Bean, wenn sie Zustand oder Abhängigkeiten hat oder im Test ersetzt werden soll.
- **`@SpringBootTest` ruft `main` nicht auf**: Es baut eine eigene `SpringApplication` ; Code in `main` läuft in Tests nicht.
- **Context-Caching**: Der Spring Context wird pro Konfiguration einmal gestartet und von allen Tests geteilt (im Log nachgezählt: ein Start für alle Tests).
- **JWKS-Ablauf**: Andere Services holen den Public Key einmal und prüfen Tokens lokal, ohne Rückfrage bei AuthKit; im Request reist nur der Token.
- **Refresh-Token ≠ Access-Token**: Beide sind gültig signiert; nur der `type_tk` -Check verhindert, dass ein 30-Tage-Token als Bearer akzeptiert wird. Jeder Service braucht diesen Check.
- **Image vs. Container, Secrets als Dateien**: Keys gehören nie ins (öffentliche) Image, sondern werden beim Container-Start eingebunden; Plattformen legen Secrets als Dateien ab. Spring versteht zusätzlich `base64:` -Locations (geprüft), passend für Railway.
- **`andReturn()` , `assertInstanceOf` , `getCause()`** : Ergebnis einer MockMvc-Anfrage weiterverwenden; Typ prüfen; die eingepackte Ursache einer Exception lesen (Exception-Chaining).
- **401 vs. 500**: 4xx heißt Fehler des Clients, 5xx Fehler des Servers. Ein abgelaufenes Cookie ist ein Client-Fall; 500 hätte das Frontend und das Monitoring in die Irre geführt.

Tests & Verifikation

Test	Ergebnis
`./mvnw verify` nach `RsaKeyFiles` -Umbau	50 Tests grün
`./mvnw verify` mit `DevKeyGeneratorTest`	59 Tests grün
`./mvnw verify` mit aufgeteilten JWKS-Tests	61 Tests grün
`./mvnw verify` nach dem 401-Fix	63 Tests grün
Jar mit leerem Key-Ordner gestartet	Keys erzeugt; zweiter Start ändert nichts; mit Schalter `false` entsteht nichts
`generate-jwt-keys.sh`	erzeugt PKCS#8/X.509, überschreibt nicht; `RsaKeyFiles` liest das Ergebnis
Wegwerf-Test `base64:` -Key-Location	Key korrekt geladen (Test danach gelöscht)
Wegwerf-Test ungültiges Refresh-Cookie	vor dem Fix 500, danach 401 per Integrationstest abgesichert
Context-Start im Integrationstest	im Log genau ein „Started BackendApplicationTests“ für alle Tests
JaCoCo	97,7 % Zeilen, 88,1 % Branches; offene Lücken notiert

Probleme & Lösungen

- **Problem:** Der Integrationstest brauchte Key-Dateien, die es in der CI nicht gibt. → **Lösung:** Keys pro Testlauf in `@TempDir` schreiben und per `@DynamicPropertySource` einspeisen.
- **Problem:** Erster Build mit `DevKeyGenerator` scheiterte an einem fehlenden Import. → **Lösung:** Import ergänzt, danach Jar-Test mit leerem Ordner.
- **Problem:** Mit `rw-------` hätte der Container-User die lokal erzeugten Dev-Keys nicht lesen können. → **Lösung:** Standardrechte für Dev-Keys; Produktions-Keys kommen aus Secrets, deren Rechte die Plattform setzt.
- **Problem:** Ungültiges Refresh-Cookie → 500. Unit- und Slice-Test waren grün, weil der Slice-Test einen Mock mit der „richtigen“ Exception benutzte. → **Lösung:** Exception im `JwtService` übersetzen und einen Integrationstest für den echten Fall ergänzen.
- **Problem:** Die Doku beschrieb noch HS256 und den alten Filter. → **Lösung:** `SECURITY.md` , `ARCHITECTURE.md` , `BACKEND_FLOW.md` neu geschrieben.

Eingesetzte Hilfsmittel

Claude Code (Anthropic). Den Integrationstest habe ich gemeinsam mit Claude aufgebaut; die JWKS-Verifikation habe ich selbst geschrieben, Claude hat sie danach in drei Tests aufgeteilt und den Refresh-Fall ergänzt. `RsaKeyFiles` , `DevKeyGenerator` , `DevKeyGeneratorTest` , das Key-Skript, der 401-Fix, die Kommentare und die Doku-Überarbeitung wurden auf meine Anfrage von Claude geschrieben; ich habe sie mir erklären lassen und geprüft. Die Commits habe ich mit meiner Git-Identität erstellen lassen. (FH-Richtlinien zur Kennzeichnung beachten.)

Offene Punkte / Nächste Schritte

- `backend-verify-ci` schloss die Integrationstests noch aus (in M05 behoben).
- Produktions-Deployment: Plattform offen (Railway mit `base64:` oder Secrets als Dateien).
- Beispiel-Microservice mit `jwk-set-uri` und `type_tk` -Validator.
- Coverage-Lücken: `DevKeyGenerator` (nur Public Key vorhanden, gemischte Locations), Schreibfehler in `RsaKeyFiles` .

Commit-Verlauf

Datum	Hash	Nachricht	Inhalt (kurz)
2026-10-11 20:14	`045b703`	refactor(auth): read, write and generate the JWT key files in RsaKeyFiles	PEM-Logik zentral in `RsaKeyFiles`, `KeyConfig` nur noch Bean-Fabrik, Test umbenannt
2026-10-11 20:14	`e827d77`	feat(auth): generate a dev JWT key pair on first start	`DevKeyGenerator`, Schalter in Properties und Dockerfile, 8 Unit-Tests
2026-10-11 20:14	`620e198`	docs: add generate-jwt-keys.sh and explain dev vs. production keys	Key-Skript, README-Schritt 0, Abschnitt „JWT keys“
2026-10-11 20:14	`fbe4fe7`	test(integration): cover the full auth flow and JWKS verification	Integrationstest mit Temp-Keys, JWKS-Struktur und -Verifikation
2026-10-11 20:14	`3aa8cb2`	test: comment the non-obvious parts of the tests	Kommentare in Unit- und Slice-Tests
2026-10-11 20:28	`b80a366`	fix(auth): answer an invalid refresh cookie with 401 instead of 500	JJWT-Fehler → `BadCredentialsException`, neue Tests
2026-10-11 20:28	`122d289`	docs: describe the RS256 resource-server flow and the current tests	Security-, Architektur- und Flow-Doku auf RS256
2026-10-11 20:35	`c54c3f6`	Merge branch 'feature/integration-test'	Merge nach `main`